

# **LAPORAN PRAKTIKUM STRUKTUR DATA**

## **SINGLE LINKED LIST**

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero



**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**PADANG**

**2026**

## **KATA PENGANTAR**

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas pada mata kuliah Praktikum Struktur Data dengan topik pembahasan mengenai struktur data Single Linked List (SLL). Dalam laporan ini dibahas konsep dasar linked list serta implementasinya melalui berbagai operasi seperti penambahan, penghapusan, pencarian, dan traversal data, termasuk penerapannya dalam studi kasus sederhana seperti sistem antrian pasien di rumah sakit. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang, serta semoga laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai struktur data linked list.

Padang, 2026

Penulis

## DAFTAR ISI

|                                  |            |
|----------------------------------|------------|
| <b>KATA PENGANTAR.....</b>       | <b>ii</b>  |
| <b>DAFTAR ISI.....</b>           | <b>iii</b> |
| <b>BAB I PENDAHULUAN.....</b>    | <b>1</b>   |
| 1.1 Latar Belakang.....          | 1          |
| 1.2 Tujuan.....                  | 1          |
| 1.3 Manfaat.....                 | 1          |
| <b>BAB II PEMBAHASAN.....</b>    | <b>2</b>   |
| 2.1 NodeSLL_2511531017.....      | 2          |
| 2.2 TambahSLL_2511531017.....    | 2          |
| 2.3 PencarianSLL_2511531017..... | 7          |
| 2.4 HapusSLL_2511531017.....     | 9          |
| <b>TUGAS.....</b>                | <b>13</b>  |
| 2.5 Pasien_2511531017.....       | 13         |
| 2.6 RumahSakit_2511531017.....   | 14         |
| <b>BAB III KESIMPULAN.....</b>   | <b>23</b>  |
| 3.1 Kesimpulan.....              | 23         |
| 3.2 Saran.....                   | 23         |
| <b>DAFTAR PUSTAKA.....</b>       | <b>24</b>  |

# **BAB I PENDAHULUAN**

## **1.1 Latar Belakang**

Struktur data merupakan bagian penting dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu struktur data yang sering digunakan adalah Single Linked List (SLL), yaitu struktur data linear yang terdiri dari kumpulan node yang saling terhubung melalui pointer atau referensi. Berbeda dengan array, linked list memiliki kelebihan dalam hal fleksibilitas ukuran karena dapat bertambah dan berkurang secara dinamis sesuai kebutuhan. Hal ini membuat SLL sangat cocok digunakan dalam berbagai kasus yang membutuhkan manipulasi data secara dinamis.

Dalam praktikum ini, dilakukan pembelajaran mengenai konsep dan implementasi Single Linked List melalui berbagai operasi dasar seperti penambahan node, penghapusan node, pencarian data, serta traversal. Selain itu, juga diterapkan studi kasus sederhana seperti sistem antrian pasien di rumah sakit untuk memahami bagaimana linked list dapat digunakan dalam kehidupan nyata. Dengan mempelajari SLL, diharapkan mahasiswa dapat memahami cara kerja struktur data ini secara lebih mendalam serta mampu mengimplementasikannya dalam penyelesaian masalah pemrograman.

## **1.2 Tujuan**

1. Memahami konsep dasar struktur data Single Linked List (SLL) beserta cara kerjanya.
2. Mampu mengimplementasikan operasi dasar pada SLL seperti penambahan, penghapusan, pencarian, dan traversal data.
3. Mengetahui penerapan SLL dalam studi kasus nyata, seperti sistem antrian pasien.

## **1.3 Manfaat**

1. Meningkatkan pemahaman mahasiswa terhadap struktur data dinamis dalam pemrograman.
2. Melatih kemampuan dalam mengelola data menggunakan pointer atau referensi secara terstruktur.
3. Memberikan gambaran penerapan linked list dalam menyelesaikan permasalahan nyata secara efisien.

## BAB II PEMBAHASAN

### 2.1 NodeSLL\_2511531017

```
package pekan5_2511531017;

public class NodeSLL_2511531017 {
    int data_1017;
    NodeSLL_2511531017 next_1017;
    public NodeSLL_2511531017(int data_1017) {
        this.data_1017 = data_1017;
        this.next_1017 = null;
    }
}
```

Kode 2.1

Penjelasan:

`int data_1017;`

Variabel ini digunakan untuk menyimpan nilai data pada setiap node dalam linked list. Tipe data yang digunakan adalah int

`NodeSLL_2511531017 next_1017;`

Variabel ini berfungsi sebagai penunjuk (reference) ke node berikutnya dalam linked list. Jika bernilai null, berarti node tersebut adalah node terakhir.

```
public NodeSLL_2511531017(int data_1017) {
    this.data_1017 = data_1017;
    this.next_1017 = null;
}
```

Constructor ini digunakan untuk membuat node baru dengan nilai data tertentu. Nilai parameter data\_1017 akan disimpan ke dalam variabel data\_1017, sedangkan next\_1017 diinisialisasi null karena node baru belum terhubung ke node lain

Class ini berfungsi sebagai pembentuk dasar struktur data Single Linked List yang menyimpan data dan referensi ke node berikutnya, sehingga memungkinkan terbentuknya rangkaian data secara dinamis.

### 2.2 TambahSLL\_2511531017

```
package pekan5_2511531017;

public class TambahSLL_2511531017 {
    public static NodeSLL_2511531017 insertAtFront(NodeSLL_2511531017
head_1017, int value_1017) {
        NodeSLL_2511531017 new_node_1017 = new
NodeSLL_2511531017(value_1017);
        new_node_1017.next_1017 = head_1017;
        return new_node_1017;
    }

    public static NodeSLL_2511531017 insertAtEnd (NodeSLL_2511531017
head_1017, int value_1017) {
        NodeSLL_2511531017 newNode_1017 = new NodeSLL_2511531017
( value_1017);
        if (head_1017 == null) {
            return newNode_1017;
        }
    }
}
```

```

    }

    NodeSLL_2511531017 last_1017 = head_1017;
    while (last_1017.next_1017 != null) {
        last_1017 = last_1017.next_1017;
    }

    last_1017.next_1017 = newNode_1017;
    return head_1017;
}

static NodeSLL_2511531017 GetNode_1017 (int data_1017) {
    return new NodeSLL_2511531017 (data_1017);
}

static NodeSLL_2511531017 insertPos (NodeSLL_2511531017 headNode_1017,
int position_1017, int value_1017) {
    NodeSLL_2511531017 head_1017 = headNode_1017;
    if (position_1017 < 1)
        System.out.print("Invalid position");
    if (position_1017 == 1) {
        NodeSLL_2511531017 new_node_1017 = new NodeSLL_2511531017
(value_1017);
        new_node_1017.next_1017 = head_1017;
        return new_node_1017;
    } else {
        while (position_1017-- != 0) {
            if (position_1017 == 1) {
                NodeSLL_2511531017 newNode_1017 =
GetNode_1017(value_1017);
                newNode_1017.next_1017 =
headNode_1017.next_1017;
                newNode_1017.next_1017 = newNode_1017;
                break;
            }
            headNode_1017 = headNode_1017.next_1017;
        }
        if (position_1017 != 1) {
            System.out.print("Posisi di luar jangkauan");
        }
        return head_1017;
    }
}

public static void printList(NodeSLL_2511531017 head_1017)
{
    NodeSLL_2511531017 curr_1017 = head_1017;
    while (curr_1017.next_1017 != null) {
        System.out.print(curr_1017.data_1017 + "--
>");

        curr_1017 = curr_1017.next_1017;
    }
    if (curr_1017.next_1017 == null) {
        System.out.print(curr_1017.data_1017);
        System.out.println();
    }
}
}

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(2);
        head_1017.next_1017 = new NodeSLL_2511531017(3);
        head_1017.next_1017.next_1017 = new NodeSLL_2511531017(5);
        head_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(6);
        System.out.print("Senarai berantai awal:");
        printList(head_1017);
        System.out.print("tambah 1 simpul di depan: ");
        int data_1017 = 1;
        head_1017 = insertAtFront(head_1017,data_1017);
        printList(head_1017);
        System.out.print("tambah 1 simpul di belakang: ");
        int data2_1017 = 7;
        head_1017 = insertAtFront(head_1017,data2_1017);
        printList(head_1017);
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_1017 = 4;
        int pos_1017 = 4;
        head_1017 = insertAtFront(head_1017, pos_1017);
        printList(head_1017);
    }
}

```

Kode 2.2

Penjelasan:

```

public static NodeSLL_2511531017 insertAtFront(NodeSLL_2511531017 head_1017,
int value_1017) {
    NodeSLL_2511531017 new_node_1017 = new
NodeSLL_2511531017(value_1017);
    new_node_1017.next_1017 = head_1017;
    return new_node_1017;
}

```

Method ini digunakan untuk menambahkan node di bagian depan linked list. Node baru dibuat, lalu `next` diarahkan ke head lama, dan node baru dikembalikan sebagai head yang baru.

```

public static NodeSLL_2511531017 insertAtEnd (NodeSLL_2511531017 head_1017,
int value_1017) {
    NodeSLL_2511531017 newNode_1017 = new NodeSLL_2511531017
( value_1017);
    if (head_1017 == null) {
        return newNode_1017;
    }

    NodeSLL_2511531017 last_1017 = head_1017;
    while (last_1017.next_1017 != null) {
        last_1017 = last_1017.next_1017;
    }

    last_1017.next_1017 = newNode_1017;
    return head_1017;
}

```

Method ini berfungsi untuk menambahkan node di bagian akhir. Jika list kosong, node baru langsung jadi head. Jika tidak, program menelusuri sampai node terakhir lalu menambahkan node baru di sana.

```
static NodeSLL_2511531017 GetNode_1017 (int data_1017) {
    return new NodeSLL_2511531017 (data_1017);
}
```

Method ini digunakan untuk membuat node baru dengan nilai tertentu. Biasanya dipakai sebagai helper agar pembuatan node lebih sederhana.

```
static NodeSLL_2511531017 insertPos (NodeSLL_2511531017 headNode_1017,
int position_1017, int value_1017) {
    NodeSLL_2511531017 head_1017 = headNode_1017;
    if (position_1017 < 1)
        System.out.print("Invalid position");
    if (position_1017 == 1) {
        NodeSLL_2511531017 new_node_1017 = new NodeSLL_2511531017
(value_1017);
        new_node_1017.next_1017 = head_1017;
        return new_node_1017;
    } else {
        while (position_1017-- != 0) {
            if (position_1017 == 1) {
                NodeSLL_2511531017 newNode_1017 =
GetNode_1017(value_1017);
                newNode_1017.next_1017 =
headNode_1017.next_1017;
                newNode_1017.next_1017 = newNode_1017;
                break;
            }
            headNode_1017 = headNode_1017.next_1017;
        }
        if (position_1017 != 1) {
            System.out.print("Posisi di luar jangkauan");
        }
        return head_1017;
    }
}
```

Method ini digunakan untuk menambahkan node pada posisi tertentu. Jika posisi 1, maka data ditambahkan di depan. Jika tidak, program akan menelusuri list sampai posisi yang dimaksud dan menyisipkan node baru. Jika posisi tidak valid, akan muncul pesan error.

```
public static void printList(NodeSLL_2511531017 head_1017) {
    NodeSLL_2511531017 curr_1017 = head_1017;
    while (curr_1017.next_1017 != null) {
        System.out.print(curr_1017.data_1017 + "-->");
        curr_1017 = curr_1017.next_1017;
    }
    if (curr_1017.next_1017 == null) {
        System.out.print(curr_1017.data_1017);
        System.out.println();
    }
}
```

Method ini berfungsi untuk menampilkan isi linked list dari awal sampai akhir dengan format panah (-->) sebagai penghubung antar node.



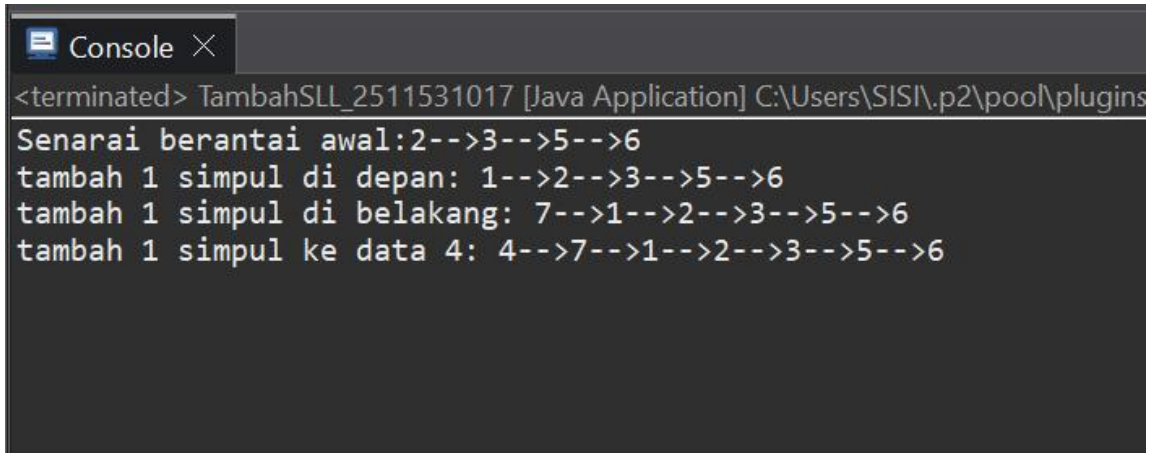
```

public static void main(String[] args) {
    // TODO Auto-generated method stub
    NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(2);
    head_1017.next_1017 = new NodeSLL_2511531017(3);
    head_1017.next_1017.next_1017 = new NodeSLL_2511531017(5);
    head_1017.next_1017.next_1017.next_1017 = new NodeSLL_2511531017(6);
    System.out.print("Senarai berantai awal:");
    printList(head_1017);
    System.out.print("tambah 1 simpul di depan: ");
    int data_1017 = 1;
    head_1017 = insertAtFront(head_1017,data_1017);
    printList(head_1017);
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_1017 = 7;
    head_1017 = insertAtFront(head_1017,data2_1017);
    printList(head_1017);
    System.out.print("tambah 1 simpul ke data 4: ");
    int data3_1017 = 4;
    int pos_1017 = 4;
    head_1017 = insertAtFront(head_1017, pos_1017);
    printList(head_1017);
}

```

Method ini digunakan untuk menguji program. Awalnya dibuat linked list dengan beberapa node, lalu dilakukan operasi penambahan di depan, belakang, dan posisi tertentu, kemudian ditampilkan hasilnya.

Output:



```

Console X
<terminated> TambahSLL_2511531017 [Java Application] C:\Users\SISI\p2\pool\plugins
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 7-->1-->2-->3-->5-->6
tambah 1 simpul ke data 4: 4-->7-->1-->2-->3-->5-->6

```

Gambar 2.2

## 2.3 PencarianSLL\_2511531017

```
package pekan5_2511531017;

public class PencarianSLL_2511531017 {
    static boolean searchKey (NodeSLL_2511531017 head_1017, int key_1017) {
        NodeSLL_2511531017 curr_1017 = head_1017;
        while (curr_1017 != null) {
            if (curr_1017.data_1017 == key_1017)
                return true;
            curr_1017 = curr_1017.next_1017;
        }
        return false;
    }

    public static void traversal (NodeSLL_2511531017 head_1017) {
        NodeSLL_2511531017 curr_1017 = head_1017;
        while (curr_1017 != null) {
            System.out.print(" " + curr_1017.data_1017);
            curr_1017 = curr_1017.next_1017;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(14);
        head_1017.next_1017 = new NodeSLL_2511531017(21);
        head_1017.next_1017.next_1017 = new NodeSLL_2511531017(13);
        head_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(30);
        head_1017.next_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(10);
        System.out.print("Penulisan SSL: ");
        traversal(head_1017);
        int key_1017 = 30;
        System.out.print("cari data " + key_1017 + " = ");
        if (searchKey (head_1017, key_1017))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

Kode 2.3

Penjelasan:

```
public class PencarianSLL_2511531017 {
    static boolean searchKey (NodeSLL_2511531017 head_1017, int key_1017) {
        NodeSLL_2511531017 curr_1017 = head_1017;
        while (curr_1017 != null) {
            if (curr_1017.data_1017 == key_1017)
                return true;
            curr_1017 = curr_1017.next_1017;
        }
        return false;
    }
}
```

Method ini digunakan untuk mencari apakah suatu nilai (key) terdapat dalam linked list. Proses dimulai dari head, lalu menelusuri setiap node satu per satu. Jika data yang dicari ditemukan, method langsung mengembalikan nilai true. Jika sampai akhir list tidak ditemukan, maka akan mengembalikan false.

```
public static void traversal (NodeSLL_2511531017 head_1017) {  
    NodeSLL_2511531017 curr_1017 = head_1017;  
    while (curr_1017 != null) {  
        System.out.print(" " + curr_1017.data_1017);  
        curr_1017 = curr_1017.next_1017;  
    }  
    System.out.println();  
}
```

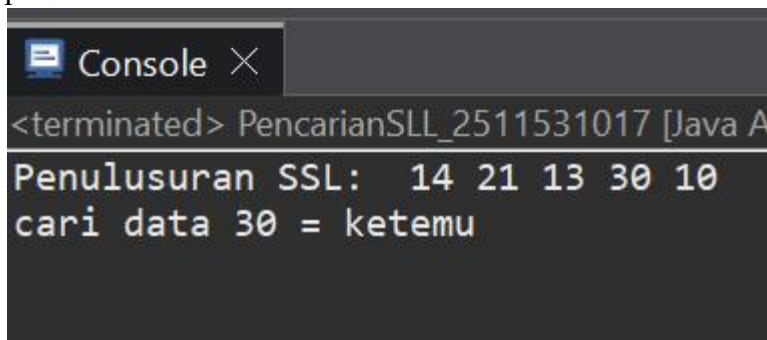
Method ini berfungsi untuk menampilkan seluruh isi linked list. Program akan menelusuri dari node pertama hingga terakhir dan mencetak setiap data yang ada.

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(14);  
    head_1017.next_1017 = new NodeSLL_2511531017(21);  
    head_1017.next_1017.next_1017 = new NodeSLL_2511531017(13);  
    head_1017.next_1017.next_1017.next_1017 = new NodeSLL_2511531017(30);  
    head_1017.next_1017.next_1017.next_1017.next_1017 = new  
NodeSLL_2511531017(10);  
    System.out.print("Penelusuran SSL: ");  
    traversal(head_1017);  
    int key_1017 = 30;  
    System.out.print("cari data " + key_1017 + " = ");  
    if (searchKey (head_1017, key_1017))  
        System.out.println("ketemu");  
    else  
        System.out.println("tidak ada");  
}
```

Method ini digunakan untuk menguji program. Pertama, dibuat linked list dengan beberapa data. Kemudian dilakukan traversal untuk menampilkan seluruh isi list. Setelah itu, program melakukan pencarian terhadap suatu nilai (misalnya 30). Jika data ditemukan, akan ditampilkan pesan "ketemu", jika tidak maka "tidak ada".

Class ini berfungsi untuk melakukan pencarian data pada linked list serta menampilkan isi list melalui proses traversal, sehingga memudahkan dalam mengetahui keberadaan suatu data dalam struktur list.

Output:



```
<terminated> PencarianSLL_2511531017 [Java A  
Penelusuran SSL: 14 21 13 30 10  
cari data 30 = ketemu
```

Gambar 2.3

## 2.4 HapusSLL\_2511531017

```
package pekan5_2511531017;

public class HapusSLL_2511531017 {
    public static NodeSLL_2511531017 deleteHead(NodeSLL_2511531017
head_1017) {
        if (head_1017 == null)
            return null;
        head_1017 = head_1017.next_1017;
        return head_1017;
    }

    public static NodeSLL_2511531017 removeLastNode (NodeSLL_2511531017
head_1017) {
        if (head_1017 == null) {
            return null;
        }
        if (head_1017.next_1017 == null) {
            return null;
        }
        NodeSLL_2511531017 secondLast_1017 = head_1017;
        while (secondLast_1017.next_1017.next_1017 != null) {
            secondLast_1017 = secondLast_1017.next_1017;
        }
        secondLast_1017.next_1017 = null;
        return head_1017;
    }

    public static NodeSLL_2511531017 deleteNode (NodeSLL_2511531017
head_1017, int position_1017) {
        NodeSLL_2511531017 temp_1017 = head_1017;
        NodeSLL_2511531017 prev_1017 = null;
        if (temp_1017 == null)
            return head_1017;
        if (position_1017 == 1) {
            head_1017 = temp_1017.next_1017;
            return head_1017;
        }
        for (int i = 1; temp_1017 != null && i < position_1017; i++) {
            prev_1017 = temp_1017;
            temp_1017 = temp_1017.next_1017;
        }
        if (temp_1017 != null) {
            prev_1017.next_1017 = temp_1017.next_1017;
        } else {
            System.out.println("Data tidak ada");
            return head_1017;
        }
        return prev_1017;
    }

    public static void printList (NodeSLL_2511531017 head_1017) {
        NodeSLL_2511531017 curr_1017 = head_1017;
        while (curr_1017.next_1017 != null) {
            System.out.print(curr_1017.data_1017+"-->");
            curr_1017 = curr_1017.next_1017;
        }
    }
}
```

```

        }
        if (curr_1017.next_1017 == null) {
            System.out.print(curr_1017.data_1017);
            System.out.println();
        }
    }

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(1);
        head_1017.next_1017 = new NodeSLL_2511531017(2);
        head_1017.next_1017.next_1017 = new NodeSLL_2511531017(3);
        head_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(4);
        head_1017.next_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(5);
        head_1017.next_1017.next_1017.next_1017.next_1017.next_1017 =
new NodeSLL_2511531017(6);
        System.out.println("list awal: ");
        printList(head_1017);
        head_1017 = deleteHead(head_1017);
        System.out.println("List setelah head dihapus: ");
        printList(head_1017);
        head_1017 = removeLastNode(head_1017);
        System.out.println("List setelah simpul terakhir dihapus: ");
        printList(head_1017);
        int position_1017 = 2;
        head_1017 = deleteNode(head_1017, position_1017);
        System.out.println("List setelah posisi 2 dihapus: ");
        printList(head_1017);
    }
}

```

Kode 2.4

Penjelasan:

```

public class HapusSLL_2511531017 {
    public static NodeSLL_2511531017 deleteHead(NodeSLL_2511531017
head_1017) {
        if (head_1017 == null)
            return null;
        head_1017 = head_1017.next_1017;
        return head_1017;
    }
}

```

Method ini digunakan untuk menghapus node pertama (head) pada linked list. Jika list kosong, akan mengembalikan null. Jika tidak, head digeser ke node berikutnya sehingga node pertama terhapus.

```

    public static NodeSLL_2511531017 removeLastNode (NodeSLL_2511531017
head_1017) {
        if (head_1017 == null) {
            return null;
        }
        if (head_1017.next_1017 == null) {
            return null;
        }
    }
}

```

```

    }
    NodeSLL_2511531017 secondLast_1017 = head_1017;
    while (secondLast_1017.next_1017.next_1017 != null) {
        secondLast_1017 = secondLast_1017.next_1017;
    }
    secondLast_1017.next_1017 = null;
    return head_1017;
}

```

Method ini berfungsi untuk menghapus node terakhir pada linked list. Program akan menelusuri hingga node kedua terakhir, lalu menghapus node terakhir dengan mengubah next menjadi null.

```

    public static NodeSLL_2511531017 deleteNode (NodeSLL_2511531017
head_1017, int position_1017) {
    NodeSLL_2511531017 temp_1017 = head_1017;
    NodeSLL_2511531017 prev_1017 = null;
    if (temp_1017 == null)
        return head_1017;
    if (position_1017 == 1) {
        head_1017 = temp_1017.next_1017;
        return head_1017;
    }
    for (int i = 1; temp_1017 != null && i < position_1017; i++) {
        prev_1017 = temp_1017;
        temp_1017 = temp_1017.next_1017;
    }
    if (temp_1017 != null) {
        prev_1017.next_1017 = temp_1017.next_1017;
    } else {
        System.out.println("Data tidak ada");
        return head_1017;
    }
    return prev_1017;
}

```

Method ini digunakan untuk menghapus node pada posisi tertentu. Program akan menelusuri list hingga posisi yang dimaksud, kemudian menghubungkan node sebelumnya dengan node setelahnya sehingga node di posisi tersebut terhapus. Jika posisi tidak ditemukan, akan ditampilkan pesan error.

```

    public static void printList (NodeSLL_2511531017 head_1017) {
        NodeSLL_2511531017 curr_1017 = head_1017;
        while (curr_1017.next_1017 != null) {
            System.out.print(curr_1017.data_1017+"-->");
            curr_1017 = curr_1017.next_1017;
        }
        if (curr_1017.next_1017 == null) {
            System.out.print(curr_1017.data_1017);
            System.out.println();
        }
    }
}

```

Method ini digunakan untuk menampilkan seluruh isi linked list dari awal hingga akhir dengan format panah(-->).

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        NodeSLL_2511531017 head_1017 = new NodeSLL_2511531017(1);
        head_1017.next_1017 = new NodeSLL_2511531017(2);
        head_1017.next_1017.next_1017 = new NodeSLL_2511531017(3);
        head_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(4);
    }
}

```

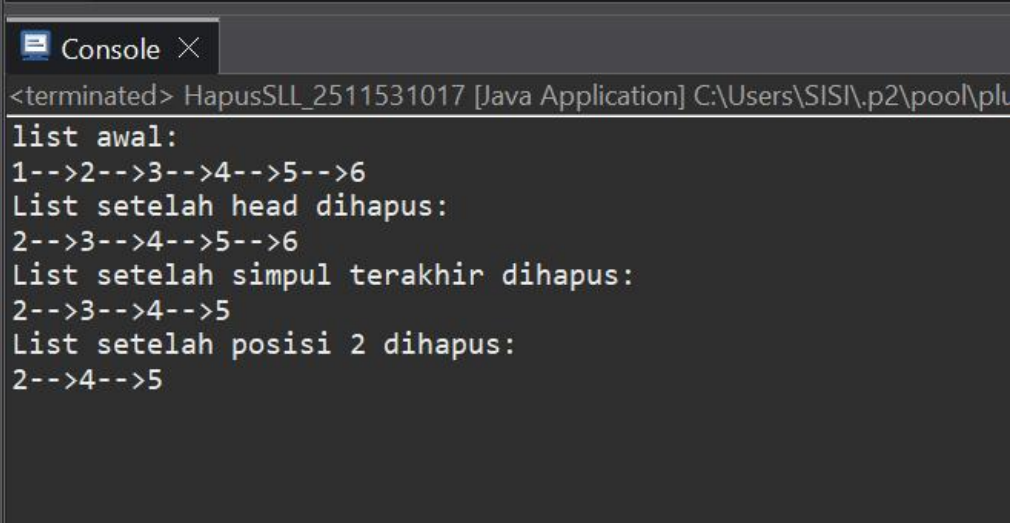
```

        head_1017.next_1017.next_1017.next_1017.next_1017 = new
NodeSLL_2511531017(5);
        head_1017.next_1017.next_1017.next_1017.next_1017.next_1017 =
new NodeSLL_2511531017(6);
        System.out.println("list awal: ");
        printList(head_1017);
        head_1017 = deleteHead(head_1017);
        System.out.println("List setelah head dihapus: ");
        printList(head_1017);
        head_1017 = removeLastNode(head_1017);
        System.out.println("List setelah simpul terakhir dihapus: ");
        printList(head_1017);
        int position_1017 = 2;
        head_1017 = deleteNode(head_1017, position_1017);
        System.out.println("List setelah posisi 2 dihapus: ");
        printList(head_1017);

```

Method ini digunakan untuk menguji operasi penghapusan pada linked list. Program membuat list awal, lalu melakukan penghapusan pada bagian depan, belakang, dan posisi tertentu. Setelah setiap operasi, list ditampilkan untuk melihat perubahan yang terjadi.

Output:



```

<terminated> HapusSLL_2511531017 [Java Application] C:\Users\SISI\p2\pool\plu
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

Gambar 2.4

## TUGAS

### 2.5 Pasien\_2511531017

```
package pekan5_2511531017;

public class Pasien_2511531017 {
    String namaPasien_1017;
    String penyakit_1017;
    int nomorAntrian_1017;
    Pasien_2511531017 next_1017;

    public Pasien_2511531017(String nama, String penyakit, int nomor) {
        this.namaPasien_1017 = nama;
        this.penakit_1017 = penyakit;
        this.nomorAntrian_1017 = nomor;
        this.next_1017 = null;
    }
    public String getNama_1017() {
        return namaPasien_1017;
    }
    public String getPenyakit_1017() {
        return penyakit_1017;
    }
    public int getNomor_1017() {
        return nomorAntrian_1017;
    }
    public Pasien_2511531017 getNext_1017() {
        return next_1017;
    }
    public void setNext_1017(Pasien_2511531017 next) {
        this.next_1017 = next;
    }
}
```

Kode 2.5

Penjelasan:

```
public Pasien_2511531017(String nama, String penyakit, int nomor) {
    this.namaPasien_1017 = nama;
    this.penakit_1017 = penyakit;
    this.nomorAntrian_1017 = nomor;
    this.next_1017 = null;
}
```

Constructor ini digunakan untuk membuat objek pasien baru dengan data nama, penyakit, dan nomor antrian. Nilai next di-set null karena node belum terhubung dengan node lain.

```
public String getNama_1017() {
    return namaPasien_1017;
}
```

Method ini digunakan untuk mengambil nilai nama pasien.

```
public String getPenyakit_1017() {
    return penyakit_1017;
}
```

Method ini digunakan untuk mengambil informasi penyakit pasien.



```
public int getNomor_1017() {
    return nomorAntrian_1017;
}
```

Method ini berfungsi untuk mengambil nomor antrian pasien.

```
public Pasien_2511531017 getNext_1017() {
    return next_1017;
}
```

Method ini digunakan untuk mengambil referensi node berikutnya dalam linked list.

```
public void setNext_1017(Pasien_2511531017 next) {
    this.next_1017 = next;
}
```

Method ini digunakan untuk menghubungkan node saat ini dengan node berikutnya.

Class ini berfungsi sebagai representasi data pasien dalam bentuk node pada linked list yang menyimpan informasi pasien dan menghubungkan antar node untuk membentuk struktur antrian pasien.

## 2.6 RumahSakit\_2511531017

```
package pekan5_2511531017;
import java.util.*;
public class RumahSakit_2511531017 {
    Pasien_2511531017 head_1017;
    int counter_1017 = 0;

    public void daftarPasien_1017(String nama, String penyakit) {
        counter_1017++;
        Pasien_2511531017 baru = new Pasien_2511531017(nama, penyakit,
counter_1017);

        if (head_1017 == null) {
            head_1017 = baru;
        } else {
            Pasien_2511531017 temp = head_1017;
            while (temp.getNext_1017() != null) {
                temp = temp.getNext_1017();
            }
            temp.setNext_1017(baru);
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
counter_1017);
    }

    public void panggilPasien_1017() {
        if (head_1017 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println("Memanggil Pasien:");
        System.out.println("Nama      : " + head_1017.getNama_1017());
        System.out.println("Keluhan  : " + head_1017.getPenyakit_1017());
        System.out.println("No Antri : " + head_1017.getNomor_1017());
        head_1017 = head_1017.getNext_1017();
    }

    public void tampilkanAntrian_1017() {
```

```

        if (head_1017 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        Pasien_2511531017 temp = head_1017;
        while (temp != null) {
            System.out.println(temp.getNomor_1017() + ". Nama: " +
temp.getNama_1017() + ". |Keluhan: " + temp.getPenyakit_1017());
            temp = temp.getNext_1017();
        }
    }

    public void cariPasien_1017(String nama) {
        Pasien_2511531017 temp = head_1017;
        boolean ketemu = false;

        while (temp != null) {
            if (temp.getNama_1017().equalsIgnoreCase(nama)) {
                System.out.println("Pasien ditemukan:");
                System.out.println(temp.getNomor_1017() + ". Nama: " +
temp.getNama_1017() + ". |Keluhan: " + temp.getPenyakit_1017());
                ketemu = true;
                break;
            }
            temp = temp.getNext_1017();
        }
        if (!ketemu) {
            System.out.println("Pasien tidak ditemukan!");
        }
    }

    public void statusAntrian_1017() {
        if (head_1017 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
        int jumlah = 0;
        Pasien_2511531017 temp = head_1017;

        while (temp != null) {
            jumlah++;
            temp = temp.getNext_1017();
        }
        System.out.println("Jumlah Pasien: " + jumlah);
        System.out.println("Pasien terdepan:");
        System.out.println("Nama: " + head_1017.getNama_1017());
        System.out.println("Keluhan: " + head_1017.getPenyakit_1017());
    }

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        RumahSakit_2511531017 rs = new RumahSakit_2511531017();
        int pilihan;
        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511531017
===");
            System.out.println("1. Daftarkan Pasien (Insert)");

```

```

        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Saerch)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan = input.nextInt();
        input.nextLine();
        switch (pilihan) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama = input.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String keluhan = input.nextLine();
                rs.daftarPasien_1017(nama, keluhan);
                break;
            case 2:
                rs.panggilPasien_1017();
                break;
            case 3:
                rs.tampilkanAntrian_1017();
                break;
            case 4:
                System.out.print("Masukkan nama yang dicari: ");
                String cari = input.nextLine();
                rs.cariPasien_1017(cari);
                break;
            case 5:
                rs.statusAntrian_1017();
                break;
            case 6:
                System.out.println("Terima kasih!");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan != 6);
    input.close();
}
}

```

Kode 2.6

Penjelasan:

```

    public void daftarPasien_1017(String nama, String penyakit) {
        counter_1017++;
        Pasien_2511531017 baru = new Pasien_2511531017(nama, penyakit,
counter_1017);

        if (head_1017 == null) {
            head_1017 = baru;
        } else {
            Pasien_2511531017 temp = head_1017;
            while (temp.getNext_1017() != null) {
                temp = temp.getNext_1017();
            }
            temp.setNext_1017(baru);
        }
    }

```

```

    }
    System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
counter_1017);

```

Method ini digunakan untuk menambahkan pasien ke dalam antrian. Nomor antrian akan bertambah otomatis setiap kali pasien baru ditambahkan. Jika antrian kosong, pasien langsung menjadi head, sedangkan jika tidak kosong, pasien akan ditambahkan di bagian akhir linked list. Proses ini memastikan urutan antrian tetap sesuai dengan prinsip FIFO. Selain itu, method ini juga menampilkan informasi bahwa pasien berhasil didaftarkan beserta nomor antriannya.

```

public void panggilPasien_1017() {
    if (head_1017 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    System.out.println("Memanggil Pasien:");
    System.out.println("Nama      : " + head_1017.getNama_1017());
    System.out.println("Keluhan   : " + head_1017.getPenyakit_1017());
    System.out.println("No Antri  : " + head_1017.getNomor_1017());
    head_1017 = head_1017.getNext_1017();
}

```

Method ini berfungsi untuk memanggil pasien paling depan (head) dalam antrian. Data pasien seperti nama, keluhan, dan nomor antrian akan ditampilkan ke layar. Setelah itu, head akan digeser ke node berikutnya sehingga pasien tersebut dianggap telah dilayani dan keluar dari antrian. Jika antrian kosong, program akan menampilkan pesan bahwa tidak ada pasien yang bisa dipanggil.

```

public void tampilkanAntrian_1017() {
    if (head_1017 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    Pasien_2511531017 temp = head_1017;
    while (temp != null) {
        System.out.println(temp.getNomor_1017() + ". Nama: " +
temp.getNama_1017() + ". |Keluhan: " + temp.getPenyakit_1017());
        temp = temp.getNext_1017();
    }
}

```

Method ini digunakan untuk menampilkan seluruh daftar pasien dalam antrian dari depan hingga belakang. Program akan melakukan traversal dari head sampai node terakhir dan mencetak data setiap pasien. Dengan method ini, pengguna dapat melihat urutan antrian secara lengkap. Jika antrian kosong, akan ditampilkan pesan bahwa tidak ada data yang tersedia.

```

public void cariPasien_1017(String nama) {
    Pasien_2511531017 temp = head_1017;
    boolean ketemu = false;

    while (temp != null) {
        if (temp.getNama_1017().equalsIgnoreCase(nama)) {
            System.out.println("Pasien ditemukan:");
            System.out.println(temp.getNomor_1017() + ". Nama: " +
temp.getNama_1017() + ". |Keluhan: " + temp.getPenyakit_1017());
            ketemu = true;
            break;
        }
        temp = temp.getNext_1017();
    }
}

```

```

    }
    temp = temp.getNext_1017();
}
if (!ketemu) {
    System.out.println("Pasien tidak ditemukan!");
}

```

Method ini digunakan untuk mencari pasien berdasarkan nama. Program akan menelusuri setiap node dalam linked list dan membandingkan nama pasien dengan input menggunakan metode equalsIgnoreCase, sehingga tidak sensitif terhadap huruf besar atau kecil. Jika pasien ditemukan, maka informasi lengkap pasien akan ditampilkan. Jika tidak ditemukan, program akan memberikan notifikasi bahwa pasien tidak ada dalam antrian.

```

public void statusAntrian_1017() {
    if (head_1017 == null) {
        System.out.println("Antrian kosong!");
        return;
    }
    int jumlah = 0;
    Pasien_2511531017 temp = head_1017;

    while (temp != null) {
        jumlah++;
        temp = temp.getNext_1017();
    }
    System.out.println("Jumlah Pasien: " + jumlah);
    System.out.println("Pasien terdepan:");
    System.out.println("Nama: " + head_1017.getNama_1017());
    System.out.println("Keluhan: " + head_1017.getPenyakit_1017());
}

```

Method ini berfungsi untuk menampilkan jumlah pasien yang sedang berada dalam antrian. Program akan menghitung jumlah node dengan melakukan traversal dari head hingga akhir. Selain itu, method ini juga menampilkan data pasien yang berada di posisi paling depan sebagai pasien berikutnya yang akan dilayani. Informasi ini penting untuk mengetahui kondisi antrian secara keseluruhan.

```

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    RumahSakit_2511531017 rs = new RumahSakit_2511531017();
    int pilihan;
    do {
        System.out.println("\n=== Antrian Rumah Sakit NIM: 2511531017  

        ===");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Saerch)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan = input.nextInt();
        input.nextLine();
    } while (pilihan != 6);
}

```

Method ini merupakan program utama yang menyediakan menu interaktif bagi pengguna. Pengguna dapat memilih berbagai operasi seperti menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, dan melihat status antrian.

Program akan terus berjalan dalam perulangan hingga pengguna memilih untuk keluar. Dengan adanya menu ini, program menjadi lebih interaktif dan mudah digunakan.

```
switch (pilihan) {
```

Struktur switch-case digunakan untuk menangani pilihan menu yang dimasukkan oleh pengguna. Setiap nilai pilihan akan menentukan operasi apa yang dijalankan dalam program antrian rumah sakit.

```
case 1:
    System.out.print("Masukkan Nama Pasien: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Keluhan: ");
    String keluhan = input.nextLine();
    rs.daftarPasien_1017(nama, keluhan);
    break;
```

Case ini digunakan untuk menambahkan pasien baru ke dalam antrian. Program akan meminta input berupa nama dan keluhan pasien, kemudian memanggil method `daftarPasien_1017()` untuk menyimpan data tersebut ke dalam linked list. Setelah itu, pasien akan mendapatkan nomor antrian secara otomatis. Proses ini memastikan data pasien tersimpan secara berurutan sesuai kedatangan.

```
case 2:
    rs.panggilPasien_1017();
    break;
```

Case ini berfungsi untuk memanggil pasien yang berada di posisi paling depan dalam antrian. Method `panggilPasien_1017()` akan menampilkan data pasien dan menghapusnya dari antrian. Proses ini sesuai dengan prinsip FIFO, di mana pasien yang pertama datang akan dilayani terlebih dahulu. Jika antrian kosong, akan ditampilkan pesan peringatan.

```
case 3:
    rs.tampilkanAntrian_1017();
    break;
```

Case ini digunakan untuk menampilkan seluruh daftar pasien yang sedang mengantri. Program akan menampilkan data pasien secara berurutan dari depan hingga belakang. Hal ini memudahkan pengguna untuk melihat kondisi antrian saat ini. Jika tidak ada data, akan ditampilkan pesan bahwa antrian kosong.

```
case 4:
    System.out.print("Masukkan nama yang dicari: ");
    String cari = input.nextLine();
    rs.cariPasien_1017(cari);
    break;
```

Case ini digunakan untuk mencari pasien berdasarkan nama. Program akan meminta input nama, kemudian memanggil method `cariPasien_1017()` untuk melakukan pencarian pada linked list. Jika ditemukan, data pasien akan ditampilkan. Jika tidak, program akan memberi informasi bahwa pasien tidak ditemukan.

```
case 5:
    rs.statusAntrian_1017();
    break;
```

Case ini berfungsi untuk menampilkan informasi jumlah pasien dalam antrian serta data pasien yang berada di posisi terdepan. Program akan menghitung jumlah node dalam linked list dan menampilkan hasilnya. Informasi ini membantu pengguna mengetahui kondisi antrian secara keseluruhan.

```
case 6:
    System.out.println("Terima kasih!");
    break;
```

Case ini digunakan untuk mengakhiri program. Ketika pengguna memilih opsi ini, program akan keluar dari perulangan dan berhenti dijalankan. Pesan penutup akan ditampilkan sebagai tanda program selesai.

```
default:
    System.out.println("Pilihan tidak valid!");
```

Bagian ini akan dijalankan jika pengguna memasukkan pilihan yang tidak sesuai dengan menu yang tersedia. Program akan memberikan peringatan agar pengguna memasukkan pilihan yang benar. Hal ini bertujuan untuk menghindari kesalahan penggunaan program.

Output:

```
=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Endy
Masukkan Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Pardilian
Masukkan Keluhan: Batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
1. Nama: Endy. |Keluhan: Demam
2. Nama: Pardilian. |Keluhan: Batuk

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari: Endy
Pasien ditemukan:
1. Nama: Endy. |Keluhan: Demam
```

```
=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah Pasien: 2
Pasien terdepan:
Nama: Endy
Keluhan: Demam
```

```
=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien:
Nama : Endy
Keluhan : Demam
No Antri : 1
```

```
=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
2. Nama: Pardilian. |Keluhan: Batuk
```

```
=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah Pasien: 1
Pasien terdepan:
Nama: Pardilian
Keluhan: Batuk
```



```

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien:
Nama      : Pardilian
Keluhan   : Batuk
No Antri  : 2

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Antrian kosong!

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama yang dicari: Endy
Pasien tidak ditemukan!

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Antrian kosong!

=== Antrian Rumah Sakit NIM: 2511531017 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Saerch)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih!

```

Gambar 2.6

## **BAB III KESIMPULAN**

### **3.1 Kesimpulan**

Single Linked List (SLL) merupakan salah satu struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung melalui referensi. Melalui praktikum ini, dapat dipahami bagaimana SLL diimplementasikan serta bagaimana operasi dasar seperti penambahan, penghapusan, pencarian, dan traversal dilakukan dalam pengelolaan data. Struktur ini memberikan fleksibilitas dalam pengolahan data karena ukurannya dapat berubah sesuai kebutuhan.

Selain itu, penerapan SLL dalam studi kasus seperti sistem antrian pasien menunjukkan bahwa struktur data ini memiliki kegunaan yang nyata dalam kehidupan sehari-hari. Dengan memahami konsep dan implementasinya, mahasiswa dapat lebih mudah mengembangkan program yang efisien dan terstruktur. Hal ini juga membantu dalam meningkatkan kemampuan logika pemrograman serta pemecahan masalah secara sistematis.

### **3.2 Saran**

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

## DAFTAR PUSTAKA

- [1] Oracle, “LinkedList Class (Java Platform SE Documentation),” [Online].  
Available: <https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>